32. Next Greater Element (NGE) in Java — Quick Notes

For every element, find the closest greater element on its RIGHT side.

Problem Statement

Given an array, find the **Next Greater Element (NGE)** for every element.

Next Greater Element

The first greater element
present on the RIGHT side.

If no greater element exists:
Output = -1

Example

Input: 5 15 10 8 6 12 7
Output: 15 -1 12 12 12 -1 -1

Understanding the Problem

For: 5 15 10 8 6 12 7

5

Right side: 15 10 8 6 12 7
First greater: 15
Output: 15

15

Right side: 10 8 6 12 7
No greater element.
Output: -1

10

Right side: 8 6 12 7
First greater: 12
Output: 12

Corner Cases

Increasing Array

10 15 20 25
Output: 15 20 25 -1

Decreasing Array

25 20 15 10
Output: -1 -1 -1 -1

Naive Solution

For each element: Move towards right until a greater element is found.

Algorithm

```
for(i=0;i<n;i++){
   next = -1;
   for(j=i+1;j<n;j++){
      if(arr[j] > arr[i]){
         next = arr[j];
         break;
      }
   }
   print(next);
}
```

Complexity

Time	Space
O(n²)	O(1)

Efficient Solution

Key Observation

If we traverse: RIGHT → LEFT
Then:

Next Greater becomes
Previous Greater.

Example: 5 15 10 8 6 12 7
Processing from right: 7 ← 12 ← 6 ← 8 ← 10 ← 15 ← 5
For `10`:
Previous Greater
(from right traversal)

= 12
which is exactly:
Next Greater
(from original array)

Stack Approach

Maintain a: Monotonic Decreasing Stack

Algorithm

For each element from RIGHT to LEFT:

Remove smaller/equal elements

```
while(!stack.isEmpty() &&
    stack.peek() <= arr[i]){
   stack.pop();
}
```

Find Next Greater

If stack empty: result[i] = -1;
Else: result[i] = stack.peek();

Push current element

stack.push(arr[i]);

Java Implementation

```
import java.util.ArrayDeque;
import java.util.Deque;

class Solution {
  static void printNGE(int arr[]) {
      int n = arr.length;
      int result[] = new int[n];

      Deque<Integer> stack = new ArrayDeque<>();
      result[n - 1] = -1;
      stack.push(arr[n - 1]);

      for(int i = n - 2; i >= 0; i--) {
          while(!stack.isEmpty() && stack.peek() <= arr[i]) {
              stack.pop();
          }
          result[i] = stack.isEmpty() ? -1 : stack.peek();
          stack.push(arr[i]);
      }

      for(int x : result) {
          System.out.print(x + " ");
      }
  }
}
```

Dry Run

Input: 5 15 10 8 6 12 7

Start

Stack: [7]

Result: _ _ _ _ _ _ -1

12

Remove:7
Stack empty.
result = -1
Push: 12

6

12 > 6

NGE:12
Push:

6
12

8
Remove:6
NGE:12
Push:

8
12

10
Remove: 8
NGE: 12
Push:

10
12

15
Remove:
10
12

Stack empty.
NGE: -1
Push: 15

5
15 > 5
NGE:15
Push:

5
15

Result: 15 -1 12 12 12 -1 -1

Why Remove Smaller Elements?

Suppose stack contains:
12
10
8
Current element: 15
All elements:
8
10
12

Are: ≤ 15

They can never become NGE again.
Remove them.

Why Result Array?

We process: RIGHT → LEFT
But output should be: LEFT → RIGHT
Hence: result[i]

stores answers.

Complexity Analysis

Time Complexity

Each element:
Pushed once
Popped at most once

Total operations: ≤ 2n
Therefore: Θ(n)

Space Complexity

Result Array: O(n)
Stack:

Worst Case: 25 20 15 10
O(n)
Total: Θ(n)

Relation with Previous Greater

Problem	Traversal Direction	Stack Stores
Previous Greater	Left → Right	Values
Next Greater	Right → Left	Values
Stock Span	Left → Right	Indexes

Interview Comment (2 lines)

// Traverse from right to left and maintain a monotonic decreasing stack.
// Remove smaller/equal elements; stack top gives the next greater element.

Interview Thinking Process

Naive:
For each element,
search right side.
↓
Repeated work exists.
↓

Need closest greater on the right.
↓

Process from right to left.
↓
Smaller elements become useless.
↓
Use Stack.

Quick Revision

Concept	Remember
Goal	Find closest greater element on right
Naive	$O(n^2)$
Efficient	Monotonic Stack
Traversal	Right → Left
Stack Type	Decreasing Stack
Store	Values
Remove	Smaller/equal elements
Empty Stack	-1
Stack Top	Next Greater
Time Complexity	$\Theta(n)$
Auxiliary Space	$\Theta(n)$